

# Documentação das mudanças realizadas no projeto ETL

## 1. Situação anterior

A versão inicial do projeto realizava um ETL com o seguinte fluxo:

Extração dos dados da Open-Meteo  
Persistência dos dados brutos no MongoDB  
Transformação dos dados em DataFrames  
Carga final em um banco SQLite local

Esse modelo já atendia ao objetivo básico de processamento de dados, porém não cumpria a exigência da disciplina, que determina o fluxo obrigatório:

API → MongoDB Atlas → Transformação → NeonDB

## 2. Requisito da disciplina

A atividade exigia que os dados fossem:

1. extraídos da API;
2. armazenados inicialmente no MongoDB Atlas;
3. recuperados do Atlas;
4. transformados;
5. carregados no NeonDB, que é um banco relacional em nuvem.

Ou seja, o projeto precisava passar a usar dois tipos de banco de dados de forma integrada:

MongoDB Atlas como banco NoSQL  
NeonDB como banco relacional

## 3. Alterações implementadas

### 3.1 Ajuste do pipeline principal

O arquivo principal [run\_etl.py](../run\_etl.py) foi ajustado para demonstrar a sequência correta da ETL:

1. Extração dos dados da API Open-Meteo

2. Persistência do conteúdo bruto no MongoDB Atlas
3. Leitura dos dados armazenados no MongoDB
4. Transformação em DataFrames com pandas
5. Salvamento final no NeonDB

### **3.2 Criação do carregamento no NeonDB**

No arquivo [src/load.py](../src/load.py), foi adicionado o método `load_neon(...)`.

Esse método:

- conecta ao PostgreSQL do NeonDB;
- cria a tabela de destino com base nos tipos das colunas do DataFrame;
- insere os registros transformados em massa;
- mantém o suporte ao SQLite local para validação, mas o fluxo principal agora é o NeonDB.

### **3.3 Atualização da documentação**

O arquivo [README.md](../README.md) foi atualizado para refletir a nova arquitetura do projeto, incluindo:

- a descrição do fluxo correto;
- o uso do MongoDB Atlas;
- o uso do NeonDB;
- as variáveis de ambiente necessárias.

### **3.4 Ajuste do ambiente**

Cada integrante deve criar seu próprio arquivo `.env` local, que permanece fora do controle de versão. Esse arquivo deve conter as duas conexões necessárias:

```
```.env
MONGODB_URI=mongodb+srv://:@.mongodb.net/?appName=
NEON_DB_URL=postgresql://:@:5432/?sslmode=require
```
```

O arquivo `.env.example` é apenas um modelo opcional e não contém credenciais reais. O arquivo `.env` está listado no `.gitignore` e não deve ser publicado.

Além disso, o arquivo [requirements.txt](../requirements.txt) foi ajustado para incluir a dependência do PostgreSQL com `psycopg2-binary`.

## 4. Impacto no projeto

Com as alterações, o projeto passou a atender ao requisito da disciplina de maneira consistente:

- extração de dados de API;
- armazenamento em banco NoSQL;
- transformação dos dados;
- armazenamento final em banco relacional em nuvem.

A arquitetura ficou alinhada ao que foi solicitado em aula e ao que foi pedido na atividade.

## 5. Validação executada

Foi realizado o teste local do pipeline com o comando:

```
```bash
python test_pipeline.py
```
```

Resultado obtido:

execução concluída com sucesso;  
mensagem final: "TODOS OS TESTES PASSARAM!"

Esse teste usa APIs simuladas, `mongomock` para representar o MongoDB Atlas e SQLite apenas como apoio à validação local. A execução de produção usa o MongoDB Atlas e o NeonDB por meio do `run\_etl.py`.

Isso confirma que a lógica do ETL continua funcionando corretamente no ambiente validado.

## 6. Conclusão

O projeto foi adaptado para cumprir a etapa de entrega solicitada pela disciplina, mantendo a estrutura orientada a objetos e ajustando o fluxo para o padrão obrigatório:

API → MongoDB Atlas → Transformação → NeonDB

Essas mudanças deixam a ETL pronta para apresentação e demonstração em aula, com os dados sendo armazenados corretamente em ambos os tipos de banco exigidos.